Oshinei

Ref. OSH-CRM-2026-002 Rev. 1
27 October 2026

To	Buzzebees Co., Ltd. — CRM PLUS project team
Cc	Easy POS — account manager and database administrator
From	Oshinei — project owner, CRM integration
Re	Your document "API CRM PLUS_full - 2026.xlsx" (10 sheets, specification versions 1–3, 2022–2025)

Proposal: integrate CRM PLUS through Oshinei's sales database, with Buzzebees as the lead party and no change to the Easy POS application

Thank you for the CRM PLUS POS API specification. Your API requires the POS application to call Buzzebees at checkout, which would require development by Easy POS. We propose a different model that needs no change to the POS application: Buzzebees reads completed sales from our Easy POS head-office database (HQ_Oshinei, Microsoft SQL Server) through read-only views, and runs point earning, reversal and coupon tracking on its own platform. This letter sets out the model, the evidence from our database, the data interface we will provide, and what we ask of Buzzebees and Easy POS.

1. Summary

Model. Buzzebees is the lead party and builds a connector that reads four read-only views in HQ_Oshinei every few minutes. Members identify a bill by claiming the receipt in the LINE Official Account. Buzzebees credits points, reverses them when a bill is voided on any later day, and marks coupons used when a cashier enters the code in Easy's existing "external coupon" function.

What changes for Easy POS. No application development. Easy is asked only for configuration: a read-only database login and views, network access, and a coupon series for Buzzebees codes (section 7).

Trade-off. Points arrive after the bill reaches the head-office database, not at the counter, and coupons are checked after use rather than before. Section 2.2 compares this with the real-time model.

Ref.	Request to Buzzebees	Priority
R1	Confirm that Buzzebees will build and operate the database connector and credit points from it	Blocking
R2	Provide a receipt-claim flow in the LINE Official Account, with the anti-abuse checks in section 6	Blocking
R3	Reverse points automatically when a claimed bill is voided, on any later day	Blocking
R4	Issue unique coupon codes usable as Easy external coupons, and mark them used from the database	Blocking
R5	Let members exchange points for coupons in LINE (no point burning at the counter)	Before pilot
R6	Follow the polling rules in section 4.3 and never write to the database	Before pilot
R7	Send a daily reconciliation report and alerts on stale branches and duplicate coupon use	Before pilot
R8	Sign a data processing agreement and store only the data listed in section 9	Before pilot
R9	Optional: credit points automatically when the cashier records the member's phone on the bill	Optional
R10	Revoke the tokens contained in the specification file and issue Oshinei credentials separately	Before pilot

We request a written reply to R1–R10 within 10 business days (proposed), followed by a joint meeting of Oshinei, Easy POS and Buzzebees.

Contents

1 Summary 1

2 Proposed integration model 4

3 Evidence from our sales database 6

4 Data interface: views, access and polling 9

5 Process flows 11

6 Requests to Buzzebees (R1–R10) 13

7 Requests to Easy POS (E1–E6) 15

8 Items Oshinei will confirm (D1–D6) 16

9 Security and data protection 17

10 Pilot, acceptance tests and next steps 18

A Appendix A — Use of the CRM PLUS API in this model 19

B Appendix B — Data sources and method 20

2. Proposed integration model

Sales flow from branch POS terminals to the head-office database as they do today. Buzzebees reads them from four read-only views and runs all loyalty logic on its own platform. The only link back to the shop is the coupon code a cashier types into Easy.

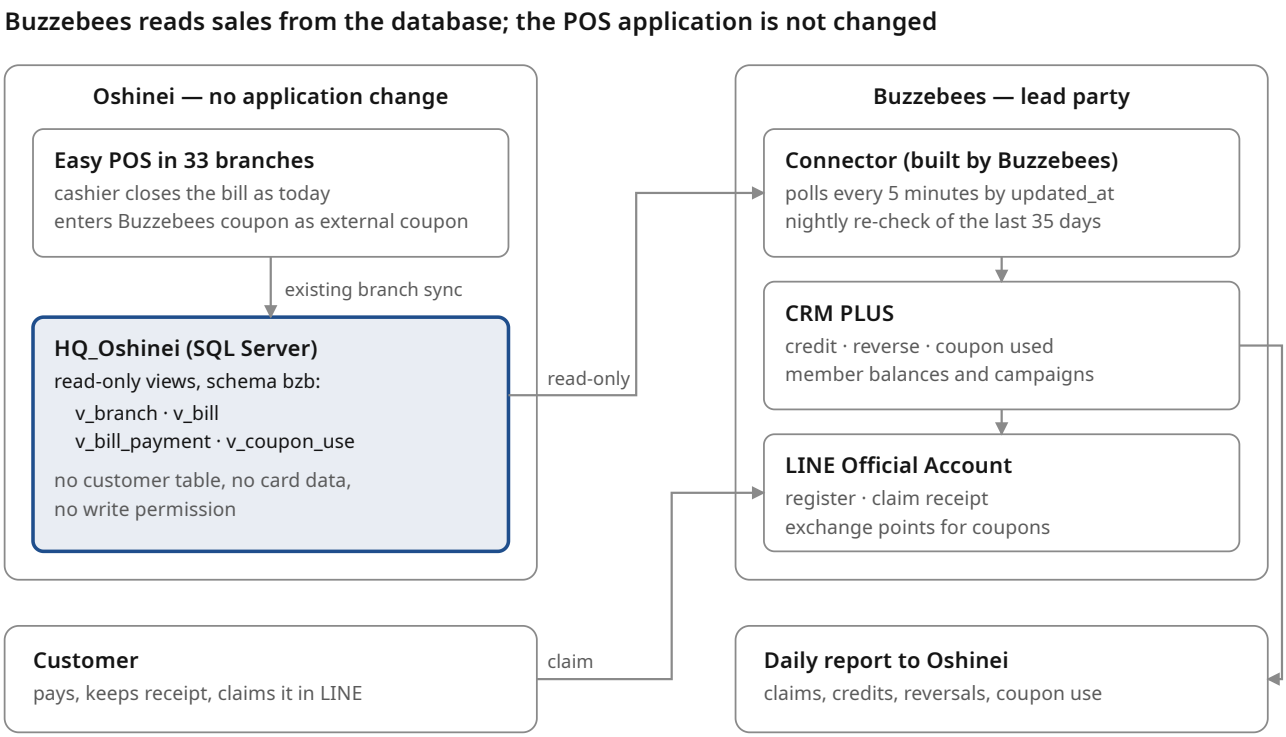


Figure 1. Proposed model. Buzzebees builds the connector; Oshinei and Easy provide read access only.

2.1 Responsibilities

Party	Responsibilities
Buzzebees	Build and operate the connector; receipt claim, crediting, reversal and coupon tracking in CRM PLUS; LINE Official Account; daily reconciliation report and alerts; data protection as a processor
Easy POS	Configuration only: read-only login and views, network access, a coupon series for Buzzebees codes, confirmation of the items in section 7
Oshinei	Business rules (section 8), approval of views and access, cashier instructions, daily reconciliation review, acceptance

2.2 Comparison with the real-time model in your specification

Topic	Database model (this proposal)	Real-time model (POS calls CRM PLUS)
Development	Buzzebees only; Easy configuration	Easy must develop the POS side through HO_CRM
Points available	After the bill reaches HQ and the member claims it (minutes to hours)	Immediately; can be printed on the receipt
Void on a later day	Always reversed: the connector sees the void whenever it happens	`/earn/cancel` allows same-day reversal only
Coupon check	After use; duplicate use detected and reported	Before use at the counter
Point burning	In LINE, by exchanging points for a coupon	At the counter
Counter workload	None for earning; type a coupon code when used	Scan member QR or enter phone on every bill
Retry and duplicate risk	None at checkout; one credit per bill key	Needs a unique reference on earn (not in the current API)

The real-time model remains possible later. The views proposed here would still be needed for reconciliation, so this work is not wasted if we move to real time.

3. Evidence from our sales database

Figures from HQ_Oshinei, January–September 2026 unless stated. They size the data Buzzebees will read and justify the rules in sections 4–6.

3.1 Volume

500,905
finalised bills, Jan–Sep 2026

≈1,650
bills on an average day (Sep 2026)

452
bills in the busiest hour (12 Aug 2026, 13:00)

33
trading branches (Sep 2026), one POS terminal each in most

Month 2026	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep
Finalised bills	69,475	63,003	50,381	54,507	61,337	47,754	50,806	54,314	49,328
Trading branches	27	27	27	29	30	31	32	33	33
Voided bills	295	233	263	254	258	275	251	317	337

Dinner is the peak: 262 bills an hour on an average day at 20:00
Average bills closed per hour, all branches, September 2026 (hour of payment)

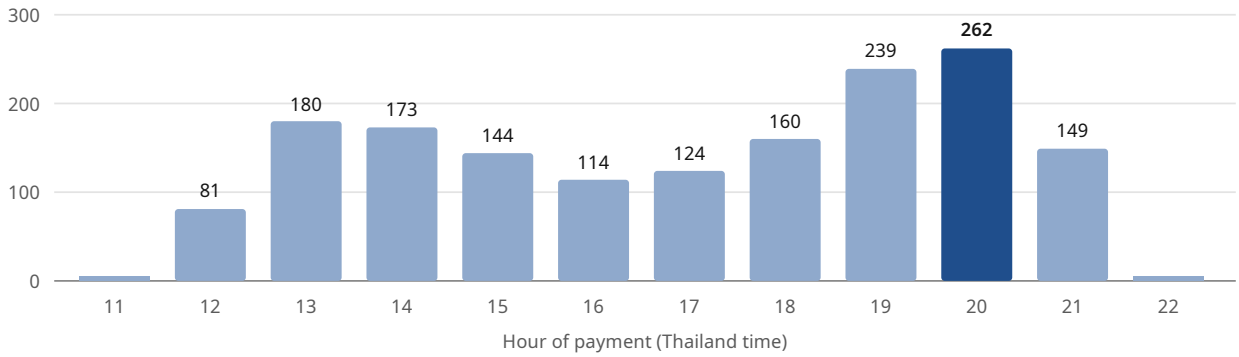


Figure 2. Average hourly bill volume, September 2026.

Implication for polling. At the busiest hour, a 5-minute poll returns about 40 new or changed bills. The load on the database is small if the connector reads only changed rows (section 4.3).

3.2 No loyalty data exists today

No bill in 2026 carries points (`Sale.EarnPoint` and `Sale.RedeemedPoint` are zero on all 500,905 bills and on all bills of 1–26 October). The Easy membership tables (`MemberCard`, `CustPoint`, `CustomerLoyalty`, `LoyaltyRank` and others) are empty. The `Customer` table (7,128 rows) holds tax-invoice records, not members. The member base therefore starts in CRM PLUS with no migration.

3.3 How a bill is identified

In September 2026 the 49,665 finalised bills had 49,665 unique branch + `SaleNum` pairs but only 33,542 distinct `SaleId` values. The bill key in this proposal is therefore **branch code + SaleNum**. `SaleNum` is printed on the receipt, contains the business date and a daily sequence, and is predictable; section 6 explains why a claim must also match the amount.

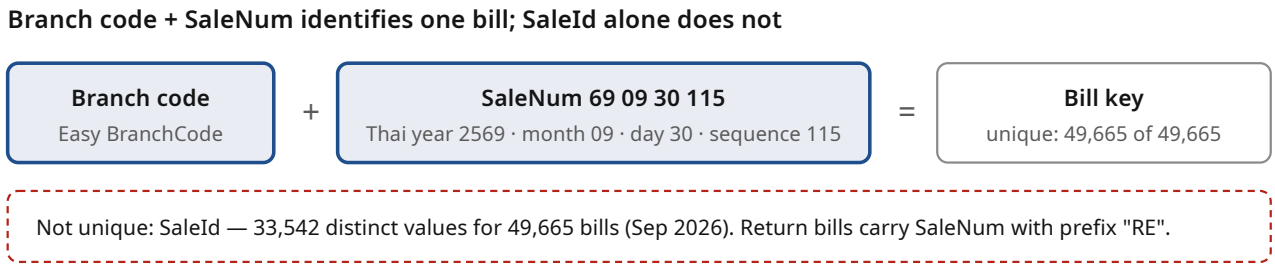


Figure 3. Bill identifier in Easy POS, used as the claim and credit key in this proposal.

3.4 Voids happen up to 19 days after the sale

Of 2,482 voided bills, 2,446 (98.5%) were voided on the day of sale and **36 on a later day, up to 19 days after**. Every voided bill updates `Sale.UpdateDate` to the void time or later (2,481 of 2,482), so the connector finds voids by reading changed rows. A credit given on day 0 can therefore always be reversed, unlike the same-day limit of `/earn/cancel`.

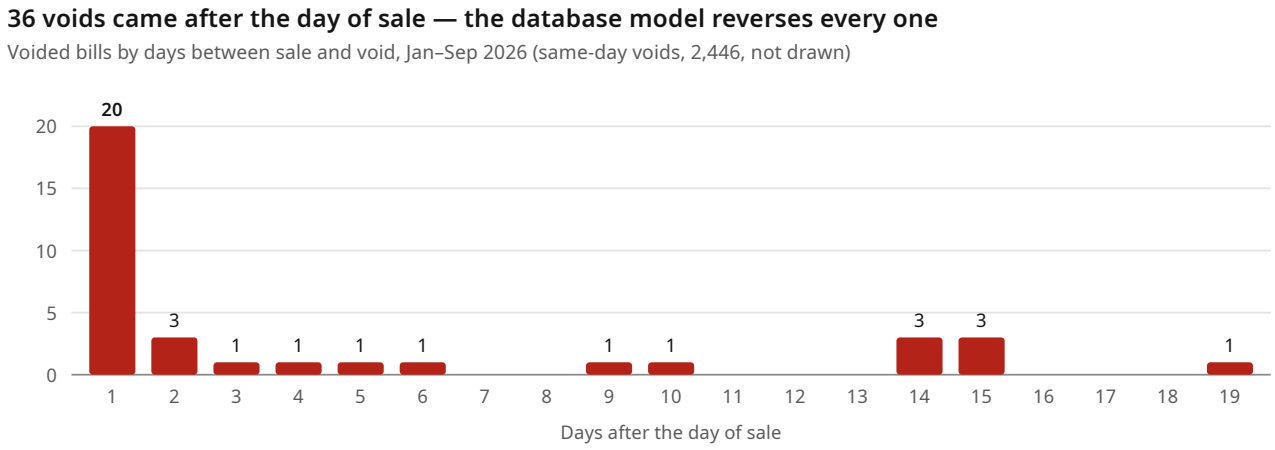


Figure 4. Voids made after the day of sale, by delay in days. Total 36; the longest delay was 19 days.

3.5 Data reaches the head office during the day

Easy records each branch's last upload in `Branch.LastSync`. On 27 October 2026 at 11:17, 16 of 31 active branches had uploaded within the previous 30 minutes, 14 within the previous 24 hours, and 1 had not uploaded for more than a day. A bill closed at 10:38 was already readable at 11:13. Upload timing differs by branch, so we will measure it for one week during the pilot and agree a target with Easy (E6). The connector should alert when a branch has not uploaded for 3 hours during trading (R7).

16 / 31	**14 / 31**	**1 / 31**	**35 min**
branches uploaded in the last 30 minutes (27 Oct, 11:17)	uploaded within the last 24 hours	no upload for more than 24 hours	bill 10:38 readable at HQ by 11:13

3.6 External coupons are already used at the counter

Cashiers already enter external coupon numbers into Easy. They are stored in `CouponCardOutsideSystemUsed` with the coupon series, coupon number, bill, branch, discount and time. From January to September 2026 there were **38,156 uses of 20,459 distinct coupon numbers**. **280 numbers were used on more than one bill** (4,506 uses in total), because only 3 of Easy's 136 coupon series enforce unique numbers (`IsUniqueCouponNumber`). On 26 October 2026, coupon number 2499 of one series was used on two bills in the same branch 77 minutes apart. Buzzebees codes must therefore be unique and checked after use (R4).

4. Data interface: views, access and polling

Oshinei will provide four read-only views in a dedicated schema `bzb`. Column names below are proposed; Easy will confirm the source columns (E1). Buzzebees receives no access to base tables.

4.1 View `bzb.v_bill` — one row per finalised or voided bill

Column	Type	Source	Meaning
bill_key	varchar(40)	BranchCode + '-' + SaleNum	Unique bill identifier; key for claims and credits
branch_code	varchar(20)	Branch.BranchCode	Easy branch code
sale_num	varchar(30)	Sale.SaleNum	Number printed on the receipt
business_date	date	Sale.SaleDate	Date of the sale
closed_at	datetime	Sale.SaleDate	Time the bill was paid (Thailand time)
terminal_id	int	Sale.TerminalId	POS terminal
guests	int	Sale.NumOfCust	Number of guests
subtotal	decimal(12,2)	Sale.SubTotal	Amount before discount and VAT (Easy to confirm)
discount	decimal(12,2)	Sale.Discount	Bill discount
service_charge	decimal(12,2)	Sale.ServiceChargeAmount	Service charge
vat	decimal(12,2)	Sale.Tax	VAT
total	decimal(12,2)	Sale.Total	Amount paid, including VAT
status	varchar(10)	Sale.Active, IsReturn	paid, voided or return
voided_at	datetime	Sale.DeleteDate	Time of void, if any
member_ref	varchar(30)	Sale.CustRefNum	Member phone if recorded by the cashier (only if R9 is adopted)
updated_at	datetime	Sale.UpdateDate	Last change; used for incremental reads

Filter: `Sale.IsFinalize = 1`; bills from 1 November 2026 (or the pilot start) onwards; pilot branches only during the pilot.

4.2 Supporting views

View	Columns	Purpose
bzb.v_branch	branch_code, branch_name, active, last_upload_at (Branch.LastSync)	Branch list for campaigns; stale-branch alerts (R7)
bzb.v_bill_payment	bill_key, payment_method, amount, updated_at	Optional earn rules by payment method. Card numbers, holder names and CVV columns in SalePayment are excluded
bzb.v_coupon_use	bill_key, coupon_series_code, coupon_number, discount_amount, used_at	Coupons entered by cashiers, filtered to Buzzebees coupon series only (R4)

A view of bill items (item code, quantity, amount) can be added later if campaigns need item-level rules (D6).

4.3 Access and polling rules

- **Account:** one SQL Server login for Buzzebees with `SELECT` on schema `bzb` only. No access to base tables, no write, no stored procedures.
- **Network:** encrypted connection (TLS) from fixed Buzzebees IP addresses on an allow-list, or through a site-to-site VPN; the exact method to be agreed with Easy (E2).
- **Incremental read:** every 5 minutes, read rows with `updated_at` later than the last watermark minus 10 minutes, to catch rows committed late. Keys are idempotent, so rows read twice cause no double credit.
- **Nightly check:** once a night, between 02:00 and 05:00, re-read the last 35 days to catch any change missed during the day.
- **Limits:** one connection at a time; queries must filter on `updated_at` or `business_date`; no full scans during trading hours (10:00–22:00).
- **No writes:** Buzzebees never writes to HQ_Oshinei. Results stay in CRM PLUS and reach Oshinei through the daily report.

5. Process flows

5.1 Earning by receipt claim

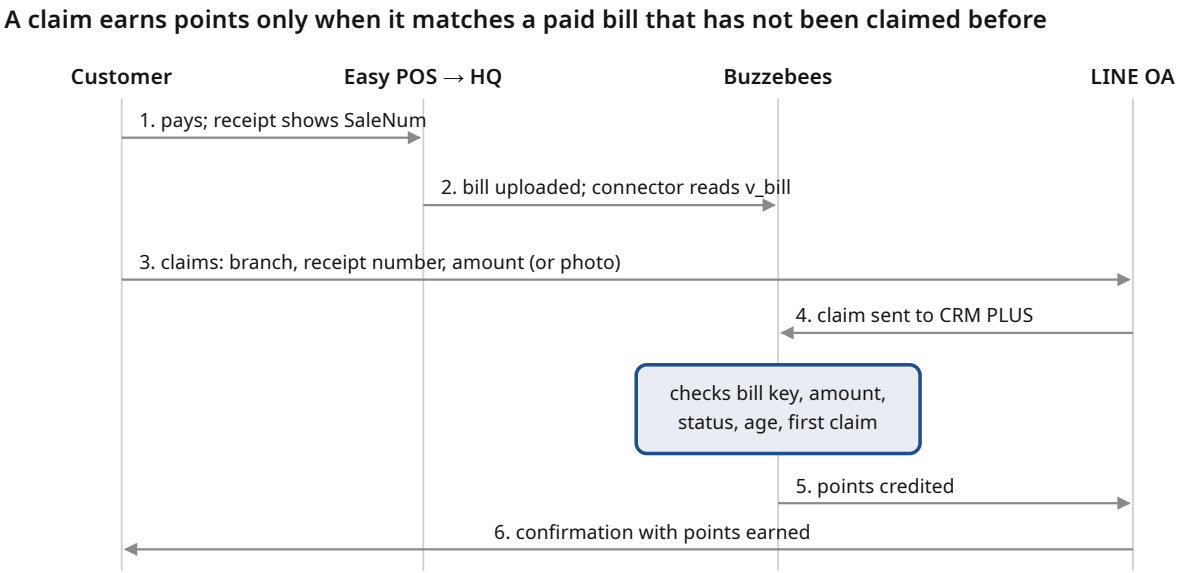


Figure 5. Receipt claim. If the bill has not reached HQ yet, the claim is held and re-checked automatically for up to 48 hours.

1. The customer pays as today. The receipt shows the branch and receipt number (`SaleNum`).
2. The bill reaches HQ_Oshinei through Easy's existing branch upload and appears in `bzb.v_bill`.
3. In the LINE Official Account the member enters the branch, receipt number and amount paid, or uploads a photo of the receipt.
4. Buzzebees checks that the bill exists, is `paid`, matches the amount, is within the claim window and has not been claimed. If the bill is not yet in HQ, the claim is held and re-checked for up to 48 hours.
5. CRM PLUS credits the points according to the earn rules (D2) and records the bill key against the credit.
6. The member receives a confirmation in LINE.

5.2 Reversal when a bill is voided

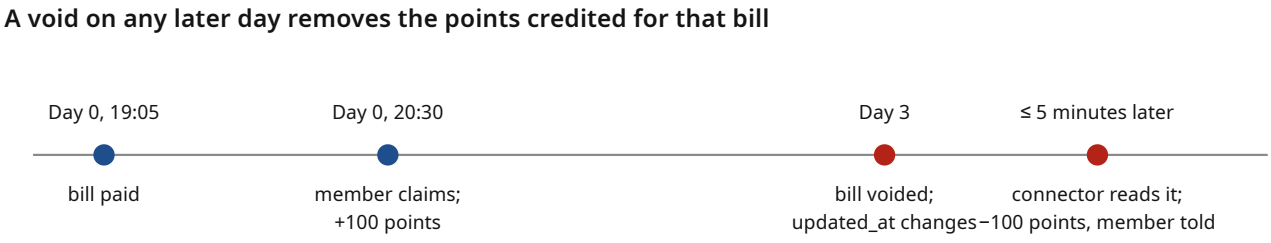


Figure 6. Reversal timeline. In 2026 the latest void was 19 days after the sale; the nightly 35-day re-check covers this.

If the member has already spent the points, CRM PLUS applies the rule chosen by Oshinei (D4): a negative balance, or deduction from the next credit. A bill that becomes `return` is treated the same way.

5.3 Coupon lifecycle

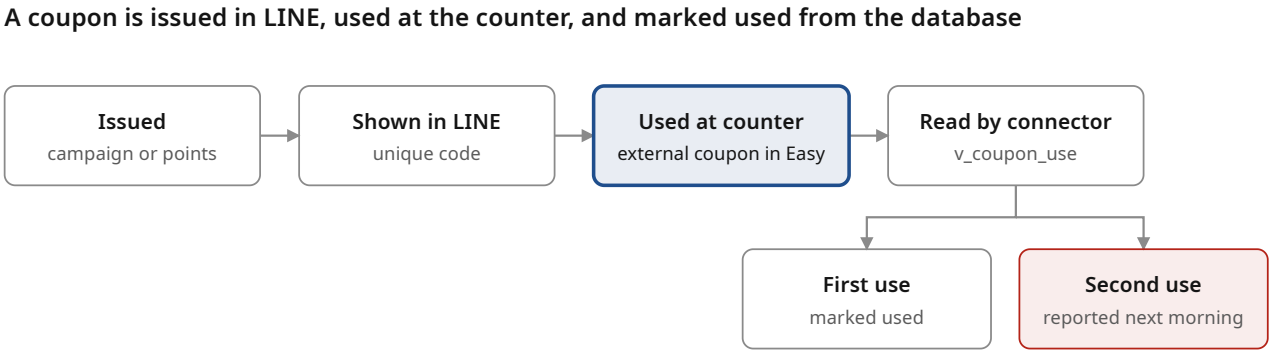


Figure 7. Coupon lifecycle in the database model.

1. CRM PLUS issues a coupon with a unique number, from a campaign or in exchange for points (R5).
2. The member shows the coupon in LINE. The cashier applies it in Easy as an external coupon of the Buzzebees series and types the number.
3. The connector reads the use from `bzb.v_coupon_use` and marks the coupon used in CRM PLUS (for example by calling `/wallet/pay` with `to_use=true` from the back end).
4. A second use of the same number is not blocked at the counter, so it is reported in the next daily report with the branch and bill. Oshinei follows it up with the branch.
5. If the bill is voided, the connector restores the coupon to unused.

Because use is checked after the event, we will limit Buzzebees coupons to modest values during the pilot (D5) and ask Easy whether the coupon series can enforce unique numbers at the counter (E3).

6. Requests to Buzzebees (R1–R10)

R1 — Build and operate the database connector Blocking

Request	Confirm that Buzzebees will build, host and operate the connector that reads the `bzb` views and credits, reverses and marks coupons in CRM PLUS. Please state the CRM PLUS functions it will use (Appendix A), the effort, the cost and the delivery date.
Why	This makes Buzzebees the single lead party and removes POS development by Easy.

R2 — Receipt claim in the LINE Official Account Blocking

Request	A claim screen that accepts branch, receipt number and amount paid, or a receipt photo. A claim succeeds only if all of the following hold: • the bill key exists in `v_bill` with status `paid`; • the amount matches `total` within 1 THB; • the business date is within the claim window (7 days proposed, D3); • the bill key has not been claimed before; • the member has made no more than 5 claims that day (proposed). Claims for bills not yet in HQ are held and re-checked for up to 48 hours.
Why	`SaleNum` is sequential within a day (section 3.3), so a receipt number alone could be guessed. The amount match and daily limit prevent claiming other customers' bills.

R3 — Automatic reversal on void Blocking

Request	When a claimed bill changes to `voided` or `return`, reverse the credit within one polling cycle, on any day, and notify the member. Re-reading the same change must not reverse twice.
Why	36 bills in 2026 were voided after the day of sale, up to 19 days later (section 3.4). `/earn/cancel` in your specification allows same-day reversal only.

R4 — Coupon codes for Easy external coupons Blocking

Request	Issue coupons with unique numeric codes of a fixed length that a cashier can type into Easy (length to be agreed with Easy, E3). Mark a coupon used when it appears in `v_coupon_use`; restore it when the bill is voided; report every second use of a code with branch, bill and time.
Why	280 external coupon numbers were used on more than one bill in 2026, because Easy enforces unique numbers in only 3 of 136 series (section 3.6).

R5 — Exchanging points for coupons in LINE Before pilot

Request	Members burn points only in LINE, by exchanging them for a coupon (R4). No point burning at the counter. Please confirm the exchange catalogue is configurable by Oshinei.

R6 — Polling discipline Before pilot

Request	Follow section 4.3: `SELECT` on `bzb` views only, incremental reads every 5 minutes, a nightly 35-day check outside trading hours, one connection, fixed source IP addresses. Provide the IP addresses and a technical contact for incidents.

R7 — Daily report and alerts Before pilot

Request	A daily file or API by 06:00 for the previous day: claims (accepted, held, rejected with reason), credits, reversals, coupons used, duplicate coupon uses, and branches with no upload for 3 hours during trading. Mismatches raised by Oshinei to be resolved within 3 business days.

R8 — Data processing agreement Before pilot

Request	Sign a data processing agreement with Oshinei as controller. Store from our database only bill keys, amounts, dates, branch codes, coupon uses and, if R9 is adopted, the member phone. Delete or anonymise sales data older than 24 months (proposed). State where data is hosted.

R9 — Optional: earning from a phone recorded on the bill Optional

Request	If Easy confirms the cashier can type the member's phone into an existing field (`CustRefNum` , empty on all 2026 bills), credit points automatically for a registered member with matching phone, without a claim.
Note	This adds counter work and needs the member's consent in LINE. Oshinei will decide after the pilot (D1).

R10 — Credentials in the specification file Before pilot

Request	The specification file contains merchant tokens in clear text, including tokens labelled "TMN only" and "All Except TMN". Please confirm whether they are valid in any environment, revoke them if so, and send Oshinei credentials (if any are needed) through a separate secure channel.

7. Requests to Easy POS (E1–E6)

Configuration only; no change to the POS application.

Ref.	Request	Why
E1	Create schema `bzb` with the four views in section 4, and confirm the source column for each field (in particular `SubTotal`, `Total`, `Discount` and the status logic)	Oshinei's own account is read-only and cannot create views
E2	Create a read-only login for Buzzebees limited to schema `bzb`, and agree network access (IP allow-list or VPN) to the HQ server	Secure access without base-table rights
E3	Set up a coupon series for Buzzebees codes as external coupons; confirm the code length accepted; confirm whether `IsUniqueCouponNumber` blocks reuse at the counter	Duplicate use today (section 3.6)
E4	Confirm that the receipt prints the branch name and `SaleNum` legibly; optionally enable the receipt QR feature already assigned to 35 branches	Members claim from the receipt
E5	Confirm whether the cashier screen allows entering a value into `CustRefNum` without development	Input to R9
E6	Confirm how often each branch uploads to HQ and whether the interval can be shortened to 15 minutes	Points arrive only after upload (section 3.5)

8. Items Oshinei will confirm (D1–D6)

Ref.	Decision	Current preference
D1	Member identification	Receipt claim first; phone on bill (R9) after the pilot
D2	Earn base and rate: `total` including VAT and service charge, or amount before VAT; whether Grab and LINE MAN bills earn	To be confirmed
D3	Claim window	7 days from the business date
D4	Reversal after points are spent: negative balance or deduction from the next credit	To be confirmed
D5	Maximum coupon value during the pilot	To be confirmed
D6	Whether item-level campaigns are needed (adds a bill-item view)	Not in the pilot

9. Security and data protection

Area	Rule
Least privilege	`SELECT` on `bzb` views only; no base tables, no write, no execute
Data excluded	`Customer` table, cashier names (`Users`), card number, holder name, CVV and expiry fields in `SalePayment`, bill notes
Network	Encrypted connection; fixed Buzzebees IP addresses or VPN; access can be suspended by Oshinei at any time
Credentials	Exchanged through a separate secure channel; rotated every 90 days (proposed); never in documents, URLs or logs
Personal data	Member identity stays in CRM PLUS. The phone is read from the bill only if R9 is adopted with member consent. Buzzebees acts as processor under the Personal Data Protection Act B.E. 2562 (R8)
Audit	Buzzebees logs every query (time, view, rows) and provides the log on request

10. Pilot, acceptance tests and next steps

10.1 Acceptance tests (pilot branch)

- [ ] A paid bill claimed with the correct amount earns points once; a second claim of the same bill is rejected.
- [ ] A claim with a wrong amount, an expired date or an unknown receipt is rejected with a clear reason.
- [ ] A claim made before the bill reaches HQ is held and credited automatically once the bill arrives.
- [ ] A claimed bill voided on day 0 and another voided on day 3 are both reversed within one polling cycle.
- [ ] A coupon used once is marked used; a second use of the same code appears in the next daily report; a coupon on a voided bill is restored.
- [ ] Re-reading the same rows (watermark overlap and nightly check) causes no duplicate credit or reversal.
- [ ] The daily report for the test week matches `v_bill` and `v_coupon_use` exactly.
- [ ] A branch with no upload for 3 hours during trading raises an alert.

10.2 Next steps

No.	Step	Owner	Timing (proposed)
1	Written reply to R1–R10 with effort, cost and date	Buzzebees	Within 10 business days
2	Written reply to E1–E6	Easy POS	Within 10 business days
3	Joint meeting; approve views, access and business rules	Oshinei (convenor)	After steps 1–2
4	Views, login and network access in place	Easy POS	After the meeting
5	Connector build and tests against a copy or the pilot branch	Buzzebees	To be agreed
6	Pilot in one branch for 4 weeks, upload timing measured in week 1, go/no-go	Oshinei	To be agreed

Yours faithfully,

For Oshinei	Acknowledged by Buzzebees
Name:	Name:
Position: Project owner, CRM integration	Position:
Date:	Date:

Appendix A — Use of the CRM PLUS API in this model

The endpoints in your specification are not called by the POS in this model. Buzzebees may use them internally from the connector; that choice is Buzzebees'.

Endpoint (your sheet)	Use in this model	Note
`GET /pos/profile` (Profile)	Not used	Members are identified in LINE, not at the counter
`POST /pos/earn_direct` (Earn via user id)	Possible, by connector	Credit a known member after a successful claim; needs a reference to prevent double credit
`POST /pos/earn` (Earn)	Possible, by connector	For R9 only (credit by phone)
`POST /earn/cancel` (Refund point)	Not sufficient	Same-day limit; R3 needs reversal on any day
`POST /wallet/pay` (Redeem, Check code)	Possible, by connector	Mark a coupon used after it appears in `v_coupon_use`
Error Code, url	For Buzzebees	Internal to the connector

Observations on the specification file (for example the `points` field missing from the Earn-via-user-id table but present in its sample, and the Earn-direct-point sample calling `/earn_direct`) are not repeated here because the POS no longer calls these endpoints.

Appendix B — Data sources and method

Item	Detail
Source	Easy POS head-office database HQ_Oshinei (Microsoft SQL Server), Oshinei read-only account, queried 14 and 27 October 2026
Period	1 January – 30 September 2026 unless stated; points checked to 26 October 2026; upload status at 11:17 on 27 October 2026
Finalised bill	`IsFinalize = 1 AND Active = 1 AND ISNULL(IsReturn,0) = 0`
Voided bill	`IsFinalize = 1 AND Active = 0`; delay = calendar days from `SaleDate` to `DeleteDate`; `UpdateDate ≥ DeleteDate` on 2,481 of 2,482
Hourly load	Hour of `SaleDate` (payment time), September 2026 average; busiest hour over July–September
Bill identifier	Distinct (BranchId, SaleNum) and distinct SaleId over finalised bills, September 2026
Upload status	`Branch.LastSync` for active branches with sales since 20 October 2026
External coupons	`CouponCardOutsideSystemUsed`, `UsedDate` in January–September 2026; reuse = same series and number on more than one bill; `CouponSeries.IsUniqueCouponNumber`

Prepared by Oshinei. This revision replaces Ref. OSH-CRM-2026-002 dated 27 October 2026, which proposed the real-time model.